Programmation Frontend

Lab 5 — Next.js App Router

Routing par fichiers — composants serveur et client

ESP/UCAD — 2025–2026

Dr. El Hadji Bassirou TOURÉ
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Informations pratiques

Durée estimée	2h (une séance)
Prérequis	Connaître React : composants, props, `useState`, `useEffect`, CSS Modules, formulaires contrôlés.
Projet	**Nouveau** projet `annuaire-next/`
Livrables	Annuaire complet sous Next.js avec 4 pages et composants serveur/client
Validation	L'application fonctionne sur `http://localhost:3000`

Objectifs du lab

À l'issue de ce lab, chaque étudiant sera capable de :

1. Expliquer **pourquoi** un framework comme Next.js est utile
2. Créer un projet Next.js et comprendre la structure `app/`
3. Créer des routes par **structure de fichiers** (pas de code de routing)
4. Utiliser `layout.js` pour partager une interface commune entre les pages
5. Distinguer un **composant serveur** d'un **composant client** et savoir quand utiliser chacun
6. Utiliser `loading.js` pour afficher un loading automatique

Table des matières

1 Pourquoi Next.js ? 3
1.1 Les limites de React seul . . . 3
1.2 Ce que Next.js change . . . 3

2 Partie 1 — Créer le projet 4
2.1 La commande . . . 4
2.2 La structure du projet . . . 4
2.3 Le routing par fichiers . . . 5

3 Partie 2 — Le concept serveur / client 5

4 Partie 3 — Styles globaux et Layout **7**
4.1 Fichier `src/app/globals.css` . . . 7
4.2 Fichier `src/app/layout.js` — Le layout racine . . . 7
4.3 Fichier `src/components/Navbar.jsx` . . . 8
4.4 Fichier `src/components/Navbar.module.css` . . . 9

5 Partie 4 — La page d'accueil **10**
5.1 Fichier `src/app/page.js` . . . 10
5.2 Fichier `src/app/page.module.css` . . . 11

6 Partie 5 — Les composants réutilisables **12**
6.1 Fichier `src/components/SearchBar.jsx` . . . 12
6.2 Fichier `src/components/SearchBar.module.css` . . . 12
6.3 Fichier `src/components/UserCard.jsx` . . . 13
6.4 Fichier `src/components/UserCard.module.css` . . . 13

7 Partie 6 — La page de liste (composant client) **15**
7.1 Fichier `src/app/users/page.js` . . . 15
7.2 Fichier `src/app/users/page.module.css` . . . 17

8 Partie 7 — Le loading automatique **18**

9 Partie 8 — La page de détail (composant serveur) **18**
9.1 Fichier `src/app/users/[id]/page.js` . . . 18
9.2 Fichier `src/app/users/[id]/page.module.css` . . . 21

10 Partie 9 — La page d'ajout (composant client) **22**
10.1 Fichier `src/app/users/add/page.js` . . . 22
10.2 Fichier `src/app/users/add/page.module.css` . . . 25

11 Arborescence finale **27**

12 Aide-mémoire **28**

13 Résumé et conclusion **29**

1 Pourquoi Next.js ?

Objectif — 10 minutes

Comprendre les limites de React seul et ce que Next.js apporte.

1.1 Les limites de React seul

Avec React (Vite), quand un utilisateur ouvre votre site, voici ce qui se passe :

1. Le navigateur télécharge une page HTML **presque vide** (juste un `<div id="root">`).
2. Le navigateur télécharge **tout le JavaScript** de l'application.
3. Le JavaScript s'exécute et **construit** la page dans le navigateur.

Conséquences :

- **SEO** : les moteurs de recherche (Google) voient une page vide. Le contenu n'existe qu'après l'exécution du JS.
- **Premier chargement lent** : l'utilisateur voit un écran blanc tant que le JS n'a pas fini de s'exécuter.
- **Routing manuel** : il faut installer React Router et configurer chaque route dans le code.

1.2 Ce que Next.js change

Avec Next.js, le serveur **pré-rend** le HTML avant de l'envoyer au navigateur. L'utilisateur voit immédiatement le contenu, sans attendre le JavaScript.

Aspect	React seul (Vite)	Next.js
HTML envoyé	Vide (`<div id="root">`)	Complet (contenu pré-rendu)
SEO	Mauvais (page vide)	Bon (contenu visible)
Premier affichage	Après le chargement du JS	Immédiat
Routing	Manuel (React Router)	Automatique (par fichiers)
Loading	À gérer avec `useState`	`loading.js` automatique
Layouts	À gérer manuellement	`layout.js` intégré

Analogie : le restaurant

React seul : le client reçoit la recette et les ingrédients, puis cuisine lui-même dans sa propre cuisine. Il ne mange qu'après avoir tout préparé.
Next.js : le serveur prépare le plat en cuisine et l'envoie prêt à manger. Le client peut manger immédiatement.

Next.js

Next.js n'est **pas** un remplacement de React. C'est un **framework** construit **au-dessus** de React. Il utilise toujours les composants React, le JSX, `useState`, `useEffect`... mais il ajoute un serveur qui pré-rend les pages et gère le routing automatiquement.

2 Partie 1 — Créer le projet

Objectif — 10 minutes

Créer un projet Next.js et comprendre sa structure de dossiers.

2.1 La commande

```
npx create-next-app@latest annuaire-next
```

L'outil pose plusieurs questions. Répondez comme suit :

Question	Réponse
TypeScript ?	**No**
ESLint ?	No
Tailwind CSS ?	**No** (on utilise CSS Modules)
`src/` directory ?	**Yes**
App Router ?	**Yes**
Turbopack ?	No
Import alias ?	Défaut (`@/*`)

Puis créez les dossiers pour les futures pages :

```
cd annuaire-next
mkdir -p src/app/users/add
mkdir -p "src/app/users/[id]"
mkdir -p src/components
```

Lancez le serveur de développement :

```
npm run dev
```

Ouvrez `http://localhost:3000`. La page par défaut de Next.js s'affiche. Le port est **3000** (Vite utilisait 5173).

2.2 La structure du projet

Voici les fichiers importants générés par Next.js :

```
annuaire-next/
  src/
    app/
      layout.js      <-- le layout racine
      page.js        <-- la page d'accueil (/)
      globals.css    <-- les styles globaux
  package.json
```

Les fichiers spéciaux de Next.js

Dans le dossier `app/`, certains noms de fichiers ont un rôle particulier :

Fichier	Rôle
`page.js`	Le contenu de la page pour cette route. C'est ce que l'utilisateur voit.
`layout.js`	L'interface **partagée** entre les pages (navbar, footer). Elle enveloppe les pages enfants et reste en place pendant la navigation.
`loading.js`	L'écran de chargement affiché **automatiquement** par Next.js pendant que la page se prépare.

2.3 Le routing par fichiers

C'est le changement le plus visible par rapport à React Router. Avec Next.js, la structure des dossiers **définit** les routes. Pas de code à écrire.

La structure des dossiers

Fichier créé	URL générée
`src/app/page.js`	`/`
`src/app/users/page.js`	`/users`
`src/app/users/[id]/page.js`	`/users/3` ou `/users/7`
`src/app/users/add/page.js`	`/users/add`

Le dossier `[id]` (avec les crochets) est un **segment dynamique**. Il correspond à n'importe quelle valeur dans l'URL. Si l'utilisateur visite `/users/3`, Next.js charge `app/users/[id]/page.js` et rend la valeur `3` accessible.

React Router vs Next.js

	React Router	Next.js
Définir une route	Dans le code (`<Route>`)	Par les dossiers/fichiers
Paramètre	`:id`	`[id]` (nom du dossier)
Lire le paramètre	`useParams()`	`params` (prop)
Lien	`<Link to="/x">`	`<Link href="/x">`
Redirection	`useNavigate()`	`useRouter().push()`

3 Partie 2 — Le concept serveur / client

Objectif — 10 minutes

Comprendre la distinction fondamentale de Next.js avant de coder.

C'est **le** concept clé de ce lab. Il faut le comprendre **avant** de créer les fichiers.
Dans Next.js, il existe deux types de composants :

	Composant serveur	**Composant client**
Par défaut ?	**Oui** (rien à écrire)	Non (il faut `"use client"`)
Où s'exécute-t-il ?	Sur le serveur	Dans le navigateur
Peut utiliser `useState` ?	Non	Oui
Peut utiliser `useEffect` ?	Non	Oui
Peut utiliser `onClick` ?	Non	Oui
Peut faire `await fetch()` ?	Oui (directement)	Non (il faut `useEffect`)
JS envoyé au navigateur ?	Aucun	Oui

Analogie : le cuisinier et le serveur

Imaginez un restaurant. Le **cuisinier** (composant serveur) prépare le plat en cuisine — le client ne le voit pas travailler, il reçoit juste le résultat. Le **serveur de salle** (composant client) interagit directement avec le client — il prend la commande, répond aux questions, gère les changements.
Un composant qui affiche des données sans interactivité = cuisinier (serveur). Un composant avec des boutons, des champs de saisie, des clics = serveur de salle (client).

La règle de décision

Posez-vous cette question pour chaque composant :

- Ce composant utilise `useState`, `useEffect`, `onClick`, `onChange` ou un autre hook ? → **Composant client** : ajoutez `"use client"` en première ligne.
- Ce composant affiche juste du contenu sans interactivité ? → **Composant serveur** : n'écrivez rien de spécial.

Dans notre annuaire, voici la répartition :

Composant	**Type**	**Pourquoi ?**
`layout.js`	Serveur	Affiche la Navbar et enveloppe les pages
`page.js` (accueil)	Serveur	Juste du texte et des liens
`users/page.js` (liste)	Client	`useState` (recherche, tri, favoris) + `useEffect` (fetch)
`users/[id]/page.js`	Serveur	Fetch avec `await` directement (pas de hooks)
`users/add/page.js`	Client	Formulaire (`useState`, `onChange`)
`users/loading.js`	Serveur	Juste du HTML
`Navbar.jsx`	Client	`usePathname()` pour le lien actif
`SearchBar.jsx`	Client	`onChange` sur l'input
`UserCard.jsx`	Client	`onClick` sur le bouton étoile

4 Partie 3 — Styles globaux et Layout

Objectif — 10 minutes

Mettre en place le reset CSS et le layout avec la Navbar.

4.1 Fichier `src/app/globals.css`

Remplacez tout le contenu de ce fichier par un reset CSS propre. Ce fichier est importé une seule fois dans `layout.js` et s'applique à **toutes** les pages :

```
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: -apple-system, BlinkMacSystemFont,
    "Segoe UI", Roboto, sans-serif;
  color: #2d3436;
  background: #f5f6fa;
  line-height: 1.6;
}

a {
  text-decoration: none;
  color: inherit;
}
```

4.2 Fichier `src/app/layout.js` — Le layout racine

Le layout est le « cadre » de l'application. Il contient ce qui est **commun à toutes les pages** : la Navbar et la balise `<main>`. Quand l'utilisateur navigue, seul le contenu à l'intérieur de `children` change. Le layout reste en place.

Remplacez tout le contenu de `src/app/layout.js` :

```
import "./globals.css";
import Navbar from "@/components/Navbar";

export const metadata = {
  title: "Annuaire Étudiants",
  description:
    "Gestion des utilisateurs - ESP/UCAD",
};

export default function RootLayout({ children }) {
  return (
    <html lang="fr">
      <body>
        <Navbar />
        <main style={{
          maxWidth: "700px",
          margin: "0 auto",
          padding: "2rem 1rem",
        }}>
```

```
          {children}
        </main>
      </body>
    </html>
  );
}
```

Quelques points à noter :

- `metadata` : Next.js utilise cet objet pour générer les balises `<title>` et `<meta description>` dans le HTML. C'est pour le SEO.
- `children` : c'est la page courante. Next.js injecte automatiquement le bon `page.js` selon l'URL.
- `@/components/Navbar` : le `@/` est un **alias** qui pointe vers le dossier `src/`. Au lieu d'écrire `"../../components/Navbar"` (chemin relatif fragile), on écrit `"@/components/Navbar"` (chemin absolu depuis `src/`).

`@/` remplace les chemins relatifs

Sans alias	Avec alias
`"../../components/Navbar"`	`"@/components/Navbar"`
`"../../../lib/api"`	`"@/lib/api"`

L'alias est configuré automatiquement lors de la création du projet Next.js. Il fonctionne partout.

4.3 Fichier `src/components/Navbar.jsx`

La Navbar est un **composant client** car elle utilise `usePathname()` — un hook qui lit l'URL courante pour mettre en surbrillance le lien actif.

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import styles from "./Navbar.module.css";

const Navbar = () => {
  const pathname = usePathname();

  const linkClass = (href) => {
    const isActive = href === "/"
      ? pathname === "/"
      : pathname.startsWith(href);
    return isActive
      ? `${styles.link} ${styles.active}`
      : styles.link;
  };

  return (
    <nav className={styles.nav}>
      <div className={styles.container}>
        <Link href="/" className={styles.logo}>
          Annuaire
        </Link>
        <div className={styles.links}>
```

```
          <Link href="/users"
                classNameName={linkClass("/users")}>
            Liste
          </Link>
          <Link href="/users/add"
                classNameName={linkClass("/users/add")}>
            + Ajouter
          </Link>
        </div>
      </div>
    </nav>
  );
};

export default Navbar;
```

Quelques différences avec React Router :

- `Link` vient de `"next/link"` (pas de `react-router-dom`).
- On utilise `href` au lieu de `to`.
- Au lieu de `NavLink` avec `isActive`, on utilise `usePathname()` pour savoir quelle page est active et appliquer la classe CSS correspondante.

4.4 Fichier `src/components/Navbar.module.css`

Un **CSS Module** est un fichier CSS classique dont les noms de classes sont rendus uniques automatiquement. On importe les classes avec `import styles from "./..."` et on les utilise avec `className={styles.nav}`.

```
.nav {
  background: #2d3436;
  padding: 0 1rem;
  position: sticky;
  top: 0;
  z-index: 10;
}

.container {
  max-width: 700px;
  margin: 0 auto;
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 56px;
}

.logo {
  font-size: 1.25rem;
  font-weight: 700;
  color: #fff;
}

.links { display: flex; gap: 0.25rem; }

.link {
  color: #b2bec3;
  padding: 0.4rem 0.9rem;
  border-radius: 6px;
```

```
  font-size: 0.9rem;
  transition: background 0.15s, color 0.15s;
}

.link:hover {
  color: #fff;
  background: rgba(255, 255, 255, 0.1);
}

.active {
  color: #fff;
  background: rgba(255, 255, 255, 0.15);
}
```

5 Partie 4 — La page d'accueil

Objectif — 5 minutes

Créer la page d'accueil — un composant serveur simple.

5.1 Fichier `src/app/page.js`

Remplacez le contenu. Notez : pas de `"use client"`. Cette page n'a ni `useState`, ni `onClick`. Elle affiche du texte et des liens. C'est un **composant serveur**.

```
import Link from "next/link";
import styles from "./page.module.css";

export default function Home() {
  return (
    <div className={styles.home}>
      <h1 className={styles.titre}>
        Annuaire des utilisateurs
      </h1>
      <p className={styles.description}>
        Consultez, recherchez et ajoutez des
        utilisateurs. Cette application est
        construite avec Next.js.
      </p>
      <div className={styles.actions}>
        <Link href="/users"
              className={styles.bouton}>
          Voir la liste
        </Link>
        <Link href="/users/add"
              className={styles.boutonSecondaire}>
          Ajouter un utilisateur
        </Link>
      </div>
    </div>
  );
}
```

5.2 Fichier `src/app/page.module.css`

Supprimez l'ancien `page.module.css` et remplacez-le :

```
.home {
  text-align: center;
  padding: 3rem 1rem;
}

.titre {
  font-size: 2rem;
  color: #2d3436;
  margin-bottom: 1rem;
  font-weight: 700;
}

.description {
  color: #636e72;
  font-size: 1.05rem;
  max-width: 450px;
  margin: 0 auto 2rem auto;
  line-height: 1.7;
}

.actions {
  display: flex;
  gap: 1rem;
  justify-content: center;
  flex-wrap: wrap;
}

.bouton {
  padding: 0.7rem 1.8rem;
  background: #0984e3;
  color: #fff;
  border-radius: 8px;
  font-weight: 500;
  transition: background 0.15s;
}
.bouton:hover { background: #0770c2; }

.boutonSecondaire {
  padding: 0.7rem 1.8rem;
  background: #fff;
  color: #0984e3;
  border: 1px solid #0984e3;
  border-radius: 8px;
  font-weight: 500;
}
.boutonSecondaire:hover { background: #ebf5fb; }
```

Vérification

Sauvegardez et ouvrez `http://localhost:3000`. La page d'accueil s'affiche avec le titre, la description et les deux boutons. La Navbar est en haut. Cliquez sur les liens — les pages n'existent pas encore (on les crée juste après).

6 Partie 5 — Les composants réutilisables

Objectif — 10 minutes

Créer les composants `SearchBar` et `UserCard` pour Next.js.

Ces composants sont réutilisés dans plusieurs pages. Ils vont dans le dossier `src/components/`. Les deux sont des **composants client** car ils utilisent des événements (`onChange`, `onClick`).

6.1 Fichier `src/components/SearchBar.jsx`

La barre de recherche reçoit deux **props** (données venant du parent) : `recherche` (la valeur actuelle du champ) et `onRecherche` (la fonction à appeler quand l'utilisateur tape). C'est un **input contrôlé** : sa valeur est pilotée par React.

```
"use client";

import styles from "./SearchBar.module.css";

const SearchBar = ({ recherche, onRecherche }) => {
  return (
    <div className={styles.container}>
      <input
        className={styles.input}
        type="text"
        placeholder="Rechercher un utilisateur..."
        value={recherche}
        onChange={(e) =>
          onRecherche(e.target.value)
        }
      />
    </div>
  );
};

export default SearchBar;
```

6.2 Fichier `src/components/SearchBar.module.css`

```
.container { margin: 1rem 0; }

.input {
  width: 100%;
  padding: 0.7rem 1rem;
  font-size: 0.95rem;
  border: 1px solid #dfe6e9;
  border-radius: 8px;
  background: #fff;
  transition: border-color 0.15s,
              box-shadow 0.15s;
}

.input:focus {
  outline: none;
  border-color: #0984e3;
  box-shadow: 0 0 0 3px rgba(9, 132, 227, 0.1);
```

```
}
```

6.3 Fichier `src/components/UserCard.jsx`

La carte affiche les informations d'un utilisateur. Elle contient un lien vers la page de détail (`<Link href={...}>`) et un bouton étoile pour les favoris (`onClick`). `"use client"` est nécessaire à cause de l'événement `onClick`.

```
"use client";

import Link from "next/link";
import styles from "./UserCard.module.css";

const UserCard = ({
  user, estFavori, onToggleFavori
}) => {
  return (
    <div className={styles.card}>
      <div className={styles.header}>
        <Link href={`/users/${user.id}`}
              className={styles.lien}>
          <h3 className={styles.nom}>
            {user.name}
          </h3>
        </Link>
        <button className={styles.etoile}
                onClick={() =>
                  onToggleFavori(user.id)
                }>
          {estFavori ? "\u2605" : "\u2606"}
        </button>
      </div>
      <p className={styles.info}>
        <span className={styles.labelInfo}>
          Email
        </span> {user.email}
      </p>
      <p className={styles.info}>
        <span className={styles.labelInfo}>
          Entreprise
        </span> {user.company.name}
      </p>
      <Link href={`/users/${user.id}`}
            className={styles.voir}>
        Voir le profil →
      </Link>
    </div>
  );
};

export default UserCard;
```

6.4 Fichier `src/components/UserCard.module.css`

```
.card {
```

```
  background: #fff;
  border: 1px solid #dfe6e9;
  padding: 1.25rem;
  margin: 0.75rem 0;
  border-radius: 10px;
  transition: box-shadow 0.15s, transform 0.15s;
}
.card:hover {
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
  transform: translateY(-1px);
}

.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 0.5rem;
}

.lien { text-decoration: none; color: inherit; }
.nom { margin: 0; font-size: 1.05rem; }
.nom:hover { color: #0984e3; }

.info {
  margin: 0.3rem 0;
  color: #636e72;
  font-size: 0.9rem;
}
.labelInfo {
  font-weight: 600;
  color: #2d3436;
  margin-right: 0.3rem;
}

.etoile {
  background: none;
  border: none;
  font-size: 1.4rem;
  cursor: pointer;
  color: #e67e22;
  padding: 0;
  line-height: 1;
  transition: transform 0.15s;
}
.etoile:hover { transform: scale(1.2); }

.voir {
  display: inline-block;
  margin-top: 0.6rem;
  font-size: 0.85rem;
  color: #0984e3;
  font-weight: 500;
}
.voir:hover { text-decoration: underline; }
```

7 Partie 6 — La page de liste (composant client)

Objectif — 10 minutes

Créer la page qui affiche la liste des utilisateurs avec recherche, tri et favoris.

Cette page utilise `useState` (pour la recherche, le tri, les favoris) et `useEffect` (pour charger les données au démarrage). C'est donc un **composant client**.

`useState` est un hook React qui permet de stocker une donnée dans un composant. Quand cette donnée change (via la fonction `set...`), React met à jour l'affichage automatiquement.

`useEffect` est un hook qui exécute du code **après** le rendu du composant. Avec un tableau vide `[]`, le code ne s'exécute qu'une seule fois au démarrage.

7.1 Fichier `src/app/users/page.js`

```
"use client";

import { useState, useEffect } from "react";
import UserCard from "@/components/UserCard";
import SearchBar from "@/components/SearchBar";
import styles from "./page.module.css";

export default function UserList() {
  // États du composant
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState(null);
  const [recherche, setRecherche] = useState("");
  const [triAscendant, setTriAscendant] =
    useState(true);
  const [favoris, setFavoris] = useState([]);

  // Charger les données au démarrage
  useEffect(() => {
    const charger = async () => {
      try {
        const res = await fetch(
          "https://jsonplaceholder.typicode.com/users"
        );
        if (!res.ok)
          throw new Error("Erreur serveur");
        const data = await res.json();
        setUsers(data);
      } catch (error) {
        setErreur(error.message);
      }
      setChargement(false);
    };
    charger();
  }, []);

  // Toggle favori (ajout/retrait)
  const toggleFavori = (userId) => {
    if (favoris.includes(userId)) {
      setFavoris(
        favoris.filter((id) => id !== userId)
```

```
      );
    } else {
      setFavoris([...favoris, userId]);
    }
  };

  // Affichage conditionnel
  if (chargement)
    return <p className={styles.message}>
      Chargement en cours...</p>;
  if (erreur)
    return <p className={styles.erreur}>
      Erreur : {erreur}</p>;

  // Filtrage et tri
  const usersFiltres = users
    .filter((u) =>
      u.name.toLowerCase()
        .includes(recherche.toLowerCase())
    )
    .sort((a, b) =>
      triAscendant
        ? a.name.localeCompare(b.name)
        : b.name.localeCompare(a.name)
    );

  return (
    <div>
      <h1 className={styles.titre}>
        Liste des utilisateurs
      </h1>
      <SearchBar recherche={recherche}
                 onRecherche={setRecherche} />
      <div className={styles.barre}>
        <span className={styles.compteur}>
          {usersFiltres.length} utilisateur(s)
        </span>
        {favoris.length > 0 && (
          <span className={styles.favoris}>
            {favoris.length} favori(s)
          </span>
        )}
        <button className={styles.boutonTri}
          onClick={() =>
            setTriAscendant(!triAscendant)
          }>
          {triAscendant ? "A → Z" : "Z → A"}
        </button>
      </div>
      {usersFiltres.map((user) => (
        <UserCard key={user.id} user={user}
          estFavori={favoris.includes(user.id)}
          onToggleFavori={toggleFavori} />
      ))}
    </div>
  );
}
```

7.2 Fichier `src/app/users/page.module.css`

```
.titre {
  font-size: 1.5rem;
  color: #2d3436;
  margin-bottom: 0.5rem;
}
.barre {
  display: flex;
  align-items: center;
  gap: 1rem;
  margin-bottom: 0.5rem;
  flex-wrap: wrap;
}
.compteur {
  color: #636e72;
  font-size: 0.9rem;
}
.favoris {
  color: #e67e22;
  font-weight: 600;
  font-size: 0.9rem;
}
.boutonTri {
  margin-left: auto;
  padding: 0.35rem 0.9rem;
  font-size: 0.85rem;
  cursor: pointer;
  border: 1px solid #dfe6e9;
  border-radius: 6px;
  background: #fff;
  font-weight: 500;
  transition: background 0.15s;
}
.boutonTri:hover { background: #f1f2f6; }

.message {
  color: #636e72;
  padding: 2rem 0;
  text-align: center;
}
.erreur {
  color: #d63031;
  font-weight: 600;
  padding: 1rem;
  background: #ffeaa7;
  border-radius: 8px;
  text-align: center;
}
```

8 Partie 7 — Le loading automatique

Objectif — 5 minutes

Ajouter un loading automatique sans gérer d'état.

Créez `src/app/users/loading.js` :

```
import styles from "./loading.module.css";

export default function Loading() {
  return (
    <div className={styles.container}>
      <div className={styles.spinner}></div>
      <p className={styles.texte}>Chargement...</p>
    </div>
  );
}
```

Créez `src/app/users/loading.module.css` :

```
.container {
  text-align: center;
  padding: 3rem 0;
}
.spinner {
  width: 40px;
  height: 40px;
  border: 3px solid #dfe6e9;
  border-top-color: #0984e3;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
  margin: 0 auto 1rem auto;
}
@keyframes spin {
  to { transform: rotate(360deg); }
}
.texte { color: #636e72; }
```

Ce fichier est affiché **automatiquement** par Next.js pendant que la page `users/page.js` se prépare. Pas de `useState("loading")` à gérer, pas de early return à écrire. Next.js s'en charge.

9 Partie 8 — La page de détail (composant serveur)

Objectif — 15 minutes

Créer une page de détail qui s'exécute **sur le serveur**. C'est le moment clé du lab.

Voici la page qui illustre le mieux la puissance de Next.js. **Pas de `"use client"`**, pas de `useState`, pas de `useEffect`. La fonction du composant est `async` et fait directement un `await fetch()`.

9.1 Fichier `src/app/users/[id]/page.js`

```
import Link from "next/link";
```

```
import styles from "./page.module.css";

export default async function UserDetail({
  params
}) {
  const { id } = await params;

  const res = await fetch(
    `https://jsonplaceholder.typicode.com/users/${id}`,
    { cache: "no-store" }
  );

  if (!res.ok) {
    return (
      <div className={styles.centre}>
        <p className={styles.erreur}>
          Utilisateur introuvable
        </p>
        <Link href="/users"
              className={styles.retour}>←
           Retour à la liste
        </Link>
      </div>
    );
  }

  const user = await res.json();

  return (
    <div>
      <Link href="/users"
            className={styles.retour}>←
         Retour à la liste
      </Link>
      <div className={styles.profil}>
        <div className={styles.avatar}>
          {user.name.charAt(0)}
        </div>
        <h1 className={styles.nom}>{user.name}</h1>
        <p className={styles.username}>
          @{user.username}
        </p>
      </div>
      <div className={styles.carte}>
        <h2 className={styles.section}>
          Coordonnées
        </h2>
        <div className={styles.champ}>
          <span className={styles.label}>
            Email
          </span>
          <span>{user.email}</span>
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>
            Téléphone
          </span>
          <span>{user.phone}</span>
```

```
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>
            Site web
          </span>
          <span>{user.website}</span>
        </div>
      </div>
      <div className={styles.carte}>
        <h2 className={styles.section}>Adresse</h2>
        <div className={styles.champ}>
          <span className={styles.label}>
            Ville
          </span>
          <span>{user.address.city}</span>
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>Rue</span>
          <span>
            {user.address.street},
            {user.address.suite}
          </span>
        </div>
      </div>
      <div className={styles.carte}>
        <h2 className={styles.section}>
          Entreprise
        </h2>
        <div className={styles.champ}>
          <span className={styles.label}>Nom</span>
          <span>{user.company.name}</span>
        </div>
        <div className={styles.champ}>
          <span className={styles.label}>
            Secteur
          </span>
          <span>{user.company.bs}</span>
        </div>
      </div>
    </div>
  );
}
```

Pourquoi ce composant est spécial

Comparez cette page avec la version React (Labs 3-4) :

	React (useEffect)	**Next.js (serveur)**
fetch	Dans un `useEffect`	Directement avec `await`
Loading	État `chargement` + early return	`loading.js` automatique
Erreur	État `erreur` + early return	Simple `if (!res.ok)`
Données	État `user` (`useState`)	Variable locale `user`
Ligne `"use client"`	Oui	Non (serveur par défaut)
JS envoyé au client	Oui (tout le composant)	Non (juste le HTML)

Le composant serveur est plus simple : une fonction `async` qui fait un `fetch`, vérifie la réponse, et retourne du JSX. Pas de hooks, pas d'états, pas de cycle de vie.

`cache: "no-store"`

Par défaut, Next.js met en cache les réponses `fetch` côté serveur (pour la performance). L'option `cache: "no-store"` force un nouveau fetch à chaque requête. Pour une API dont les données changent souvent, c'est nécessaire.

`params` — le paramètre d'URL

Le composant reçoit un objet `params` qui contient les segments dynamiques de l'URL. Pour `/users/3`, `params.id` vaut `"3"`. C'est l'équivalent serveur de `useParams()` côté client.

9.2 Fichier `src/app/users/[id]/page.module.css`

```
.retour {
  display: inline-block;
  color: #0984e3;
  font-size: 0.9rem;
  font-weight: 500;
  margin-bottom: 1.5rem;
}
.retour:hover { text-decoration: underline; }

.profil {
  text-align: center;
  margin-bottom: 2rem;
}
.avatar {
  width: 72px;
  height: 72px;
  border-radius: 50%;
  background: #0984e3;
  color: #fff;
  font-size: 2rem;
  font-weight: 700;
  display: flex;
  align-items: center;
  justify-content: center;
  margin: 0 auto 0.75rem auto;
}
.nom {
  font-size: 1.5rem;
  color: #2d3436;
  margin: 0;
}
.username {
  color: #636e72;
  font-size: 0.95rem;
  margin-top: 0.2rem;
}

.carte {
```

```
  background: #fff;
  border: 1px solid #dfe6e9;
  border-radius: 10px;
  padding: 1.25rem;
  margin-bottom: 1rem;
}
.section {
  font-size: 0.85rem;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  color: #636e72;
  margin: 0 0 0.75rem 0;
  font-weight: 600;
}
.champ {
  display: flex;
  justify-content: space-between;
  padding: 0.5rem 0;
  border-bottom: 1px solid #f1f2f6;
  font-size: 0.95rem;
}
.champ:last-child { border-bottom: none; }
.label {
  font-weight: 600;
  color: #2d3436;
}

.erreur {
  color: #d63031;
  font-weight: 600;
  margin-bottom: 1rem;
}
.centre {
  text-align: center;
  padding: 2rem 0;
}
```

10 Partie 9 — La page d'ajout (composant client)

Objectif — 10 minutes

Créer le formulaire d'ajout d'utilisateur.

Ce formulaire utilise `useState` (pour les champs et les erreurs), `onChange` (événements de saisie) et `useRouter().push()` (redirection après soumission). C'est un **composant client**.

Un **input contrôlé** est un champ dont la valeur est stockée dans l'état React. À chaque frappe, `onChange` met à jour l'état, et React re-rend le composant avec la nouvelle valeur. L'état React est la « source de vérité » pour le contenu du champ.

10.1 Fichier `src/app/users/add/page.js`

```
"use client";

import { useState } from "react";
```

```
import { useRouter } from "next/navigation";
import styles from "./page.module.css";

export default function AddUser() {
  const router = useRouter();
  const [form, setForm] = useState({
    name: "", email: "",
    phone: "", company: "",
  });
  const [erreurs, setErreurs] = useState({});
  const [enCours, setEnCours] = useState(false);
  const [soumis, setSoumis] = useState(false);

  // Met à jour le champ modifié
  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm({ ...form, [name]: value });
    if (erreurs[name])
      setErreurs({ ...erreurs, [name]: null });
  };

  // Vérifie que les champs requis sont remplis
  const valider = () => {
    const err = {};
    if (!form.name.trim())
      err.name = "Le nom est obligatoire";
    if (!form.email.trim())
      err.email = "L'email est obligatoire";
    else if (!form.email.includes("@"))
      err.email = "L'email doit contenir un @";
    if (!form.phone.trim())
      err.phone = "Le téléphone est obligatoire";
    return err;
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const err = valider();
    if (Object.keys(err).length > 0) {
      setErreurs(err);
      return;
    }

    setEnCours(true);
    try {
      const res = await fetch(
        "https://jsonplaceholder.typicode.com/users",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
          },
          body: JSON.stringify({
            name: form.name,
            email: form.email,
            phone: form.phone,
            company: {
              name: form.company || "Non renseigné",
```

```
            },
          }),
        }
      );
      if (!res.ok)
        throw new Error("Erreur serveur");

      setSoumis(true);
      setTimeout(
        () => router.push("/users"), 2000
      );
    } catch (error) {
      setErreurs({ submit: error.message });
    }
    setEnCours(false);
  };

  if (soumis) {
    return (
      <div className={styles.succes}>
        <div className={styles.check}>&#10003;</div>
        <h2>Utilisateur ajouté</h2>
        <p>Redirection vers la liste...</p>
      </div>
    );
  }

  return (
    <div>
      <h1 className={styles.titre}>
        Ajouter un utilisateur
      </h1>
      <div className={styles.carte}>
        <form onSubmit={handleSubmit}>
          {[
            { id: "name",
              label: "Nom complet *",
              placeholder: "Fatou Ndiaye" },
            { id: "email",
              label: "Email *",
              placeholder:
                "fatou.ndiaye@ucad.edu.sn" },
            { id: "phone",
              label: "Téléphone *",
              placeholder: "77 123 45 67" },
            { id: "company",
              label: "Entreprise",
              placeholder: "ESP/UCAD" },
          ].map((field) => (
            <div key={field.id}
                 className={styles.groupe}>
              <label className={styles.label}
                     htmlFor={field.id}>
                {field.label}
              </label>
              <input
                className={`${styles.input} ${
                  erreurs[field.id]
```

```
                    ? styles.inputErreur : ""
                }`}
                type="text"
                id={field.id}
                name={field.id}
                placeholder={field.placeholder}
                value={form[field.id]}
                onChange={handleChange}
              />
              {erreurs[field.id] && (
                <p className={styles.erreur}>
                  {erreurs[field.id]}
                </p>
              )}
            </div>
          ))}
          {erreurs.submit && (
            <p className={styles.erreurGlobale}>
              {erreurs.submit}
            </p>
          )}
          <button type="submit"
                  className={styles.bouton}
                  disabled={enCours}>
            {enCours ? "Envoi..." : "Ajouter"}
          </button>
        </form>
      </div>
    </div>
  );
}
```

useRouter vient de next/navigation

En Next.js, `useRouter` est importé de `"next/navigation"` (pas de `"next/router"` qui est l'ancienne version). La méthode `router.push("/users")` fonctionne comme `navigate("/users")` dans React Router.

10.2 Fichier `src/app/users/add/page.module.css`

```
.titre {
  font-size: 1.5rem;
  color: #2d3436;
  margin-bottom: 1.5rem;
}
.carte {
  background: #fff;
  border: 1px solid #dfe6e9;
  border-radius: 10px;
  padding: 1.5rem;
}
.groupe { margin-bottom: 1.25rem; }
.label {
  display: block;
  font-size: 0.9rem;
  font-weight: 600;
```

```
  color: #2d3436;
  margin-bottom: 0.4rem;
}
.input {
  width: 100%;
  padding: 0.65rem 0.9rem;
  font-size: 0.95rem;
  border: 1px solid #dfe6e9;
  border-radius: 8px;
  background: #fff;
}
.input:focus {
  outline: none;
  border-color: #0984e3;
  box-shadow: 0 0 0 3px rgba(9, 132, 227, 0.1);
}
.inputErreur { border-color: #d63031; }
.erreur {
  color: #d63031;
  font-size: 0.8rem;
  margin-top: 0.3rem;
}
.erreurGlobale {
  color: #d63031;
  font-weight: 600;
  padding: 0.75rem;
  background: #ffeaa7;
  border-radius: 6px;
  margin-bottom: 1rem;
}
.bouton {
  width: 100%;
  padding: 0.75rem;
  background: #0984e3;
  color: #fff;
  border: none;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
}
.bouton:hover { background: #0770c2; }
.bouton:disabled {
  background: #b2bec3;
  cursor: not-allowed;
}
.succes {
  text-align: center;
  padding: 3rem 1rem;
}
.check {
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: #00b894;
  color: #fff;
  font-size: 2rem;
  display: flex;
```

```
  align-items: center;
  justify-content: center;
  margin: 0 auto 1rem auto;
}
```

11 Arborescence finale

Votre projet contient maintenant **15 fichiers** :

```
annuaire-next/
  src/
    app/
      globals.css              (reset CSS)
      layout.js                (layout racine)
      page.js                  (accueil)
      page.module.css
      users/
        page.js                (liste - client)
        page.module.css
        loading.js              (loading auto)
        loading.module.css
        [id]/
          page.js               (détail - serveur)
          page.module.css
        add/
          page.js               (ajout - client)
          page.module.css
    components/
      Navbar.jsx                (client)
      Navbar.module.css
      UserCard.jsx              (client)
      UserCard.module.css
      SearchBar.jsx             (client)
      SearchBar.module.css
```

Chacun de ces fichiers a été donné intégralement dans ce lab.

12 Aide-mémoire

Concept	**Syntaxe**
Créer le projet	`npx create-next-app@latest mon-app`
Lancer le serveur	`npm run dev` (port 3000)
Page	`app/chemin/page.js`
Layout	`app/chemin/layout.js`
Loading	`app/chemin/loading.js`
Route dynamique	`app/[param]/page.js`
Composant serveur	Par défaut (rien à écrire)
Composant client	`"use client"` en première ligne
Link	`import Link from "next/link"` `<Link href="/users">...</Link>`
Alias import	`import X from "@/components/X"`
useRouter	`import { useRouter } from "next/navigation"`
usePathname	`import { usePathname } from "next/navigation"`
Metadata (SEO)	`export const metadata = { title: "..." }`
No cache	`fetch(url, { cache: "no-store" })`

13 Résumé et conclusion

Dans ce lab, vous avez :

1. Compris **pourquoi** Next.js : SEO, performance, routing automatique.
2. Créé un projet Next.js et appris la structure `app/`.
3. Créé des routes par **structure de fichiers** (un dossier = une URL).
4. Utilisé `layout.js` pour la Navbar commune et `loading.js` pour le chargement automatique.
5. Distingué **composants serveur** (par défaut, avec `await fetch`) et **composants client** (`"use client"`, avec hooks et événements).
6. Créé les 15 fichiers du projet (chaque composant + son CSS Module).

Étape	Statut	Ce que vous savez faire
JS moderne	✓	Fonctions fléchées, async/await
React — composants	✓	JSX, props, état, useEffect
React — avancé	✓	Router, formulaires, CSS Modules
Next.js App Router	✓	Routing fichier, layout, server/client
API Django	→ Lab 6	CORS, auth, CRUD

Et ensuite ?

L'annuaire fonctionne sous Next.js avec du routing par fichiers, des composants serveur et client, et un loading automatique. Mais les données viennent encore de JSONPlaceholder (une API de test).
Dans le **Lab 6**, vous connecterez l'annuaire à un **vrai backend Django**. Vous apprendrez CORS, l'authentification par token, et le CRUD complet avec des données persistées en base de données.

Exercice bonus

Bonus : page 404 personnalisée

Next.js fournit un fichier spécial pour les pages introuvables. Créez `src/app/not-found.js` :

```
import Link from "next/link";

export default function NotFound() {
  return (
    <div style={{
      textAlign: "center",
      padding: "3rem"
    }}>
      <h1>404</h1>
      <p>Cette page n'existe pas.</p>
      <Link href="/">Retour à l'accueil</Link>
    </div>
  );
}
```

Testez en tapant `/xyz` dans l'URL.

Lab 5 — Next.js App Router — Terminé ✓

Vous savez créer une application Next.js complète
avec routing par fichiers, layouts et composants serveur/client.

Prochaine étape : connecter à un backend Django.

Un dossier = une route. Un fichier = une page.